

使用 MediaPipe Tasks 建構手寫數字分類器 Android 應用程式

來源：<https://codelabs.developers.google.com/mp-digit-classifier-android?hl=zh-tw>

步驟數：6

瞭解如何使用圖片分類功能，透過 MediaPipe 偵測 Android 中的手寫數字。

目錄

1. [簡介](#)
2. [在 Android 應用程式中新增 MediaPipe Tasks](#)
3. [建立 MediaPipe Tasks 數字分類器輔助程式](#)
4. [使用 MediaPipe Tasks 執行推論](#)
5. [部署及測試應用程式](#)
6. [恭喜！](#)

1. 簡介

什麼是 MediaPipe ？

[MediaPipe Solutions](#) 可讓您在應用程式中套用機器學習 (ML) 解決方案。這個框架可讓您設定預先建構的處理管道，為使用者提供即時、吸引人且實用的輸出內容。您甚至可以使用 [MediaPipe Model Maker](#) 自訂這些解決方案，更新預設模型。

圖像分類是 MediaPipe Solutions 提供的其中一項 ML 視覺工作。MediaPipe Tasks 適用於 Android、iOS、Python (包括 Raspberry Pi !) 和網頁。

在本程式碼研究室中，您會先使用 Android 應用程式在畫面上繪製數字，然後新增功能，將繪製的數字分類為 0 到 9 的單一值。

課程內容

- 如何使用 [MediaPipe Tasks](#) 在 Android 應用程式中整合圖片分類工作。

軟硬體需求

- 已安裝 [Android Studio](#) 版本 (本程式碼研究室是使用 Android Studio Giraffe 編寫及測試)。
 - 用於執行應用程式的 Android 裝置或模擬器。
 - 具備 Android 開發的基本知識 (這不是「Hello World」，但也不會太難！)。
-

2. 在 Android 應用程式中新增 MediaPipe Tasks

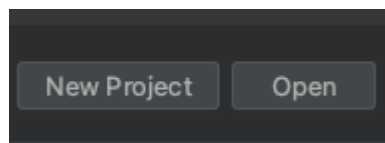
下載 Android 範例應用程式

本程式碼研究室會從預先製作的範例開始，讓您在畫面上繪圖。您可以在官方 MediaPipe 範例存放區 ([這裡](#)) 找到起始應用程式。按一下「Code」>「Download ZIP」，複製存放區或下載 zip 檔案。

- 封存檔包含與本程式碼研究室無關的其他 MediaPipe 範例。本程式碼研究室的程式碼位於 `codelabs/digitclassifier/android` 資料夾中。
- 我們已在 Android 應用程式中實作繪圖畫布，並提供樣板程式碼來擷取繪圖。原始碼位於 `codelabs/digitclassifier/android/start/` 資料夾中。
- 在本教學課程的後續步驟中，您將編寫程式碼，使用 TensorFlow Lite 模型將畫布上繪製的圖片分類為 0 到 9 的數字。
- 如果您不知道如何進行任何步驟，可以查看 `codelabs/digitclassifier/android/finish/` 資料夾中的完成應用程式。

將應用程式匯入 Android Studio

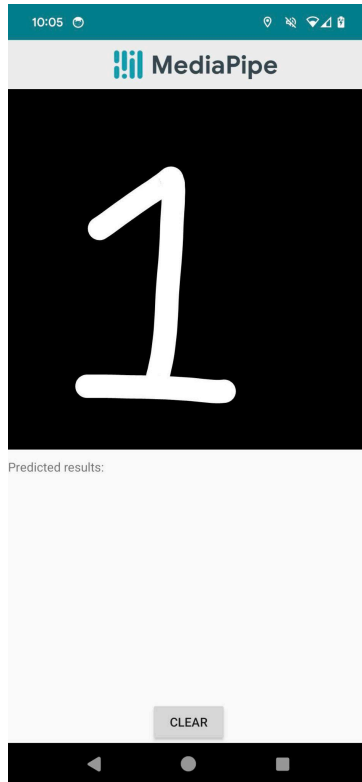
1. 開啟 Android Studio。
2. 在「Welcome to Android Studio」畫面中，選取右上角的「Open」。



3. 前往複製或下載存放區的位置，然後開啟 `codelabs/digitclassifier/android/start` 目錄。
4. 按一下 Android Studio 右上方的綠色「執行」箭頭 (



-), 確認所有項目都已正確開啟
5. 應用程式應該會開啟黑色畫面，供您繪圖，並顯示「清除」按鈕，可重設該畫面。雖然您可以在該畫面上繪圖，但除此之外就沒什麼功能了，因此我們現在要開始修正這個問題。



型號

首次執行應用程式時，您可能會發現系統下載名為 **mnist.tflite** 的檔案，並儲存在應用程式的 **assets** 目錄中。為求簡單，我們已採用可分類數字的已知模型 MNIST，並透過專案中的 **download_models.gradle** 指令碼，將模型新增至應用程式。如果您決定訓練自己的自訂模型 (例如手寫字母模型)，請移除 **download_models.gradle** 檔案，刪除應用程式層級 **build.gradle** 檔案中對該檔案的參照，然後在程式碼中變更模型名稱 (具體來說，是在 **DigitClassifierHelper.kt** 檔案中)。

更新 build.gradle

您必須先匯入程式庫，才能開始使用 MediaPipe Tasks。

1. 開啟 *app* 模組中的 **build.gradle** 檔案，然後向下捲動至 *dependencies* 區塊。
2. 您應該會在該區塊底部看到 *// STEP 1 Dependency Import* 註解。
3. 將該行替換成下列實作項目

```
implementation("com.google.mediatask:tasks-vision:latest.release")
```

4. 按一下 Android Studio 頂端橫幅中的「Sync Now」按鈕，即可下載這項依附元件。
-

3. 建立 MediaPipe Tasks 數字分類器輔助程式

在下一個步驟中，您將填入一個類別，負責處理機器學習分類的繁重工作。開啟 **DigitClassifierHelper.kt**，開始進行設定！

1. 在課程頂端找到標示「// STEP 2 Create listener」的註解
2. 將該行替換成以下程式碼。這會建立一個事件監聽器，用於將結果從 DigitClassifierHelper 類別傳遞回監聽這些結果的位置 (在本例中為 **DigitCanvasFragment** 類別，但我們很快就會介紹這個類別)

```
// STEP 2 Create listener

interface DigitClassifierListener {
    fun onError(error: String)
    fun onResults(
        results: ImageClassifierResult,
        inferenceTime: Long
    )
}
```

3. 您也需要接受 DigitClassifierListener 做為類別的選用參數：

```
class DigitClassifierHelper(
    val context: Context,
    val digitClassifierListener: DigitClassifierListener?
) {
```

4. 向下找到 *// STEP 3 define classifier* 這行，然後新增下列程式碼，為這個應用程式使用的 ImageClassifier 建立預留位置：

// STEP 3 define classifier

```
private var digitClassifier: ImageClassifier? = null
```

5. 在看到註解 *// STEP 4 set up classifier* 的位置新增下列函式：

```
// STEP 4 set up classifier
private fun setupDigitClassifier() {

    val baseOptionsBuilder = BaseOptions.builder()
        .setModelAssetPath("mnist.tflite")

    // Describe additional options
    val optionsBuilder = ImageClassifierOptions.builder()
        .setRunningMode(RunningMode.IMAGE)
        .setBaseOptions(baseOptionsBuilder.build())

    try {
        digitClassifier =
            ImageClassifier.createFromOptions(
                context,
                optionsBuilder.build()
            )
    } catch (e: IllegalStateException) {
        digitClassifierListener?.onError(
            "Image classifier failed to initialize. See error logs for " +
                "details"
        )
        Log.e(TAG, "MediaPipe failed to load model with error: " + e.message)
    }
}
```

上述章節有幾項事項，因此我們來看看較小的部分，深入瞭解實際情況。

```
val baseOptionsBuilder = BaseOptions.builder()
    .setModelAssetPath("mnist.tflite")

// Describe additional options
val optionsBuilder = ImageClassifierOptions.builder()
    .setRunningMode(RunningMode.IMAGE)
    .setBaseOptions(baseOptionsBuilder.build())
```

這個區塊會定義 ImageClassifier 使用的參數。這包括應用程式中儲存的模型 (**mnist.tflite**)，以及 BaseOptions 下的 RunningMode，在本例中為 IMAGE，但 VIDEO 和 LIVE_STREAM 也是可用的選項。其他可用參數包括 MaxResults (限制模型傳回的結果數量上限) 和 ScoreThreshold (設定模型傳回結果前必須達到的最低信賴度)。

- MNIST 模型只會傳回單一項目，因此本程式碼研究室不需要 MaxResults。
- 在本程式碼研究室中，您可以忽略 ScoreThreshold 選項，因為我們會接受模型傳回的任何單一值。

```

try {
    digitClassifier =
        ImageClassifier.createFromOptions(
            context,
            optionsBuilder.build()
        )
} catch (e: IllegalStateException) {
    digitClassifierListener?.onError(
        "Image classifier failed to initialize. See error logs for " +
        "details"
    )
    Log.e(TAG, "MediaPipe failed to load model with error: " + e.message)
}

```

建立設定選項後，您可以傳遞內容和選項，建立新的 `ImageClassifier`。如果初始化程序發生錯誤，系統會透過 `DigitClassifierListener` 傳回錯誤。

6. 由於我們會在 `ImageClassifier` 使用前初始化，因此可以新增 `init` 區塊來呼叫 `setupDigitClassifier()`。

```

init {
    setupDigitClassifier()
}

```

7. 最後，向下捲動至「// STEP 5 create classify function」的註解，並新增下列程式碼。這個函式會接受 *Bitmap* (在本例中為繪製的數字)，並將其轉換為 `MediaPipe Image` 物件 (`MPIImage`)，然後使用 `ImageClassifier` 分類該圖片，並記錄推論作業所需時間，最後透過 `DigitClassifierListener` 傳回這些結果。

```

// STEP 5 create classify function
fun classify(image: Bitmap) {
    if (digitClassifier == null) {
        setupDigitClassifier()
    }

    // Convert the input Bitmap object to an MPIImage object to run inference.
    // Rotating shouldn't be necessary because the text is being extracted from
    // a view that should always be correctly positioned.
    val mpImage = BitmapImageBuilder(image).build()

    // Inference time is the difference between the system time at the start and finish of
the
    // process
    val startTime = SystemClock.uptimeMillis()

    // Run image classification using MediaPipe Image Classifier API
    digitClassifier?.classify(mpImage)?.also { classificationResults ->
        val inferenceTimeMs = SystemClock.uptimeMillis() - startTime
        digitClassifierListener?.onResults(classificationResults, inferenceTimeMs)
    }
}

```


輔助檔案就介紹到這裡！在下一節中，您會填寫最後的步驟，開始分類繪製的數字。

4. 使用 MediaPipe Tasks 執行推論

如要開始本節，請在 Android Studio 中開啟 **DigitCanvasFragment** 類別，所有作業都會在此進行。

1. 您應該會在檔案最底部看到 *// STEP 6 Set up listener* 註解。您將在此新增與監聽器相關聯的 `onResults()` 和 `onError()` 函式。

```
// STEP 6 Set up listener
override fun onError(error: String) {
    activity?.runOnUiThread {
        Toast.makeText(requireActivity(), error, Toast.LENGTH_SHORT).show()
        fragmentDigitCanvasBinding.tvResults.text = ""
    }
}

override fun onResults(
    results: ImageClassifierResult,
    inferenceTime: Long
) {
    activity?.runOnUiThread {
        fragmentDigitCanvasBinding.tvResults.text = results
            .classificationResult()
            .classifications().get(0)
            .categories().get(0)
            .categoryName()

        fragmentDigitCanvasBinding.tvInferenceTime.text = requireActivity()
            .getString(R.string.inference_time, inferenceTime.toString())
    }
}
```

`onResults()` 特別重要，因為它會顯示從 `ImageClassifier` 收到的結果。由於這個回呼是從背景執行緒觸發，您也需要在 Android 的 UI 執行緒上執行 UI 更新。

2. 由於您要在上一步驟中從介面新增函式，因此也需要在類別頂端新增實作宣告。

```
class DigitCanvasFragment : Fragment(), DigitClassifierHelper.DigitClassifierListener
```

3. 在類別頂端，您應該會看到註解，指出 *// STEP 7a Initialize classifier*。您將在此放置 `DigitClassifierHelper` 的聲明。

```
// STEP 7a Initialize classifier.
private lateinit var digitClassifierHelper: DigitClassifierHelper
```

4. 向下移至 *// STEP 7b Initialize classifier*，您可以在 `onViewCreated()` 函式中初始化 `digitClassifierHelper`。

```
// STEP 7b Initialize classifier
// Initialize the digit classifier helper, which does all of the
// ML work. This uses the default values for the classifier.
digitClassifierHelper = DigitClassifierHelper(
    context = requireContext(), digitClassifierListener = this
)
```

5. 在最後一個步驟中，找出註解 *// STEP 8a*: classify**，並加入下列程式碼來呼叫稍後新增的函式。當您在應用程式中將手指從繪圖區域移開時，這個程式碼區塊會觸發分類作業。

```
// STEP 8a: classify
classifyDrawing()
```

6. 最後，找到註解 *// STEP 8b classify*，新增 `classifyDrawing()` 函式。這會從畫布中擷取點陣圖，然後傳遞至 `DigitClassifierHelper` 執行分類，並在 `onResults()` 介面函式中接收結果。

```
// STEP 8b classify
private fun classifyDrawing() {
    val bitmap = fragmentDigitCanvasBinding.digitCanvas.getBitmap()
    digitClassifierHelper.classify(bitmap)
}
```

步驟 5 · 約 3 分鐘

5. 部署及測試應用程式

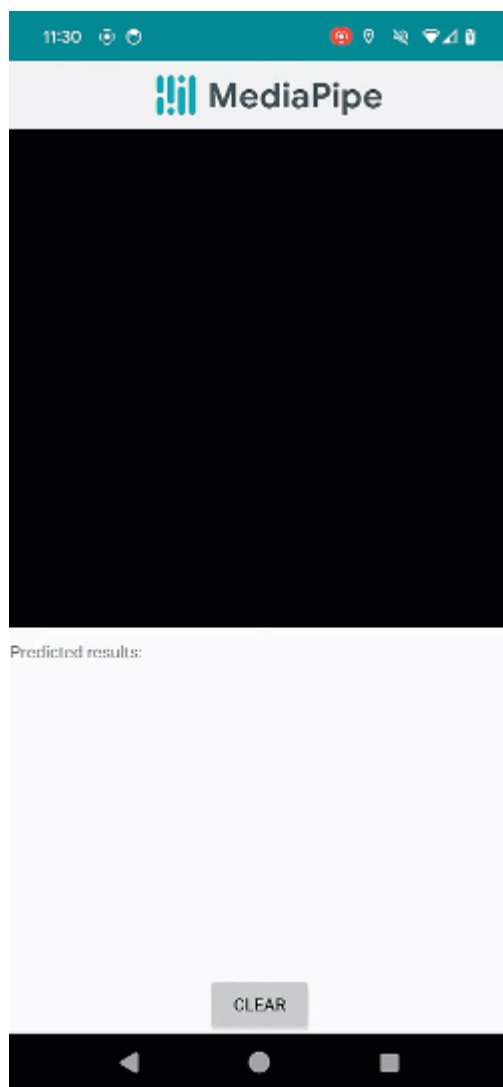
完成上述所有步驟後，您應該就能獲得可運作的應用程式，並在螢幕上分類繪製的數字！請將應用程式部署至 Android Emulator 或實體 Android 裝置，進行測試。

1. 在 Android Studio 工具列中，按一下「Run」圖示 (



) 執行應用程式。

2. 在繪圖板上畫出任何數字，看看應用程式是否能辨識。這項功能應顯示模型認為繪製的數字，以及預測該數字所花的時間。



步驟 6 · 約 0 分鐘

6. 恭喜！

你成功了！在本程式碼研究室中，您已瞭解如何將圖片分類功能新增至 Android 應用程式，以及如何使用 MNIST 模型分類手繪數字。

後續步驟

- 現在您已可分類數字，接下來不妨訓練自己的模型，分類手繪字母、動物，或無數其他項目。若要瞭解如何使用 [MediaPipe Model Maker](#) 訓練新的圖片分類模型，請參閱 developers.google.com/mediapipe 頁面上的說明文件。
- 瞭解適用於 Android 的其他 [MediaPipe Tasks](#)，包括臉部地標偵測、手勢辨識和音訊分類。

我們很期待看到你製作的各種酷炫內容！
